

# Forecasting Character Interaction Networks from Movie Screenplays: Methods and Computational Pipeline

David Brewster

June 22, 2026

## Abstract

We describe a computational pipeline for converting movie screenplays into temporal character-interaction networks, visualizing the cumulative development of those networks, and forecasting the interaction structure of the next scene. Screenplays are acquired either as HTML documents or local PDF files and parsed by a large language model into a constrained JSON representation. Character identifiers are assigned in chronological order of first speaking appearance. For visualization, scene-level interactions are accumulated through time. For prediction, however, each scene is represented by a separate undirected clique adjacency matrix, so the learning task is to infer the next local interaction matrix from a fixed window of preceding scenes. Symmetry is exploited by storing only the upper triangle of each  $256 \times 256$  matrix. A CPU-only PyTorch variational recurrent autoencoder (VRAE)-inspired model encodes five previous scene matrices into a Gaussian latent variable and decodes that latent state into Bernoulli probabilities for the next scene. Evaluation is performed on movies held out in their entirety and emphasizes precision-recall metrics because the adjacency vectors are highly sparse.

## 1 Project overview

The project has four connected stages:

1. acquire screenplay text from an online HTML source or a local PDF file;
2. parse the screenplay into a standardized movie, character, and scene representation;
3. construct and interactively visualize the temporal character-interaction network;
4. train a temporal generative model to forecast the next scene-local interaction matrix.

A central distinction is maintained between the representation used for visualization and the representation used for prediction. The visualization is cumulative: once a character or interaction has appeared, it remains visible at all later times. The forecasting model instead receives a sequence of noncumulative, scene-local matrices and predicts the local matrix of the next scene.

## 2 Screenplay acquisition

### 2.1 HTML screenplays

When a screenplay is available from the Internet Movie Script Database (IMSDB), the corresponding HTML file is downloaded and stored locally. The relevant screenplay text is extracted from the

HTML document while preserving scene headings, character cue lines, and their chronological order. The original HTML file is retained as a provenance record so that parsing can be repeated if the extraction or language-model prompt changes.

## 2.2 Local PDF screenplays

When no suitable HTML screenplay is available, a local PDF transcript or screenplay is used. Text is extracted in page order and supplied to the same downstream parser. PDF extraction may introduce artifacts such as repeated headers, page numbers, broken lines, or unusual whitespace. The language-model parsing stage is therefore instructed to identify screenplay structure semantically rather than relying only on exact HTML formatting.

## 2.3 Unified input abstraction

Both acquisition routes produce a textual screenplay representation and metadata including the movie title and source path. The subsequent parsing stage is source-agnostic: the same schema and validation rules are used for HTML-derived and PDF-derived scripts.

# 3 Large-language-model screenplay parsing

## 3.1 Output schema

A reasoning-capable large language model is prompted to return JSON only, following the schema

```
{
  "title": "Up",
  "character_ids": [
    {
      "name": "CARL",
      "id": 0
    }
  ],
  "scenes": [
    {
      "heading": "INT. MOVIE THEATRE - CONTINUOUS",
      "character_ids": [0]
    }
  ]
}
```

The `character_ids` array defines a movie-specific mapping from normalized speaking-character names to integer identifiers. The `scenes` array records scene order, the scene heading, and the identifiers of characters who have dialogue in that scene.

## 3.2 Character identifiers

Character identifiers are assigned in chronological order of first speaking appearance. Thus, character 0 is the first character with a dialogue cue, character 1 is the next previously unseen speaker, and so forth. The identifiers are local to a movie; character 0 in one movie is not assumed to represent

the same social role or identity as character 0 in another movie. Nevertheless, the chronological coordinate system is deliberately preserved because the project asks whether comparable interaction patterns emerge across films when characters are indexed by narrative introduction order.

Names are normalized consistently within a screenplay so that superficial cue variations do not create duplicate characters. Parenthetical annotations attached to a cue are not treated as new identities unless the screenplay clearly indicates a distinct character.

### 3.3 Scene membership

A character is included in a scene when the character has at least one dialogue cue in that scene. Characters who are visually present but have no line are generally not represented, because dialogue cues are the consistently observable unit across heterogeneous screenplay formats. Duplicate appearances by the same speaker within one scene are collapsed to a single identifier.

### 3.4 Validation

Every parsed file is checked before graph construction. The validator requires

- a nonempty movie title;
- integer character identifiers equal to  $0, 1, \dots, n - 1$ ;
- nonempty character names;
- scene headings represented as strings;
- scene character lists containing only valid identifiers;
- no more than  $2^8 = 256$  characters.

If any movie contains more than 256 characters, preprocessing stops before any processed dataset is written. Every offending filename, title, and character count is reported. Characters are never silently truncated or merged.

## 4 Temporal interaction networks

### 4.1 Scene-local interaction graph

For a movie with  $n \leq 256$  speaking characters, let

$$C_s \subseteq \{0, 1, \dots, n - 1\}$$

be the set of characters who speak in scene  $s$ . The local interaction matrix

$$B_s \in \{0, 1\}^{256 \times 256}$$

is defined by

$$B_s(i, j) = \begin{cases} 1, & i \in C_s \text{ and } j \in C_s, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

Consequently, the speaking characters in each scene form a clique. The matrix is symmetric. Every speaking character receives a diagonal entry  $B_s(i, i) = 1$ , including a character who speaks

alone. Off-diagonal entries represent scene-level co-occurrence rather than a directly observed exchange of dialogue between a specific pair.

Matrices are padded to  $256 \times 256$  so that every movie has the same coordinate dimension. Coordinates corresponding to nonexistent characters remain zero.

## 4.2 Cumulative graph for visualization

The interactive visualization uses the cumulative adjacency matrix

$$A_s = \bigvee_{t=0}^s B_t, \quad (2)$$

where  $\vee$  denotes entrywise logical OR. Thus, a character node is present after its first speaking scene, and an edge remains present after the first scene in which the corresponding characters co-occur.

The visualization is implemented with D3.js [1]. It provides play, pause, and scene-scrubbing controls; identifies characters on hover; and adds nodes and edges as the selected scene index advances. When the user scrubs backward, the graph is reconstructed from the prefix  $B_0, \dots, B_s$  rather than merely retaining the current state. This guarantees that nodes and edges first introduced after scene  $s$  are removed correctly.

The cumulative graph is used only as a descriptive visualization. It is not supplied to the forecasting model because monotone edge persistence would make next-step prediction dominated by copying the previous matrix.

## 5 Upper-triangular vectorization

Because  $B_s$  is symmetric, only entries with  $i \leq j$  are stored. Define

$$b_s = \text{ut}(B_s),$$

where  $\text{ut}$  packs the upper triangle, including the diagonal, into a vector. The number of retained coordinates is

$$d = \frac{256(256 + 1)}{2} = 32,896. \quad (3)$$

Thus,

$$b_s \in \{0, 1\}^{32,896}.$$

This representation exactly preserves the original matrix and reduces storage and input dimensionality by approximately one half. A deterministic inverse mapping reconstructs the full symmetric matrix for visualization and interpretation.

## 6 Forecasting samples

With a window length  $w = 5$ , the sample ending at scene  $s$  is

$$X_s = (b_{s-w+1}, \dots, b_s) \in \{0, 1\}^{w \times d}, \quad (4)$$

with target

$$y_s = b_{s+1} \in \{0, 1\}^d. \quad (5)$$

A movie containing  $T$  scenes contributes  $T - w$  sliding-window samples. Windows never cross movie boundaries.

The processed dataset stores compressed scene arrays separately for each movie. This avoids duplicating overlapping input windows on disk. A dataset loader constructs each window on demand and caches a small number of recently accessed movie arrays.

## 7 Movie-level data partition

Movies, rather than windows, are assigned to partitions. Twenty percent of movies are reserved for final testing. Ten percent of the remaining development movies are used for validation, yielding expected proportions of approximately

$$72\% \text{ training,} \quad 8\% \text{ validation,} \quad 20\% \text{ testing.}$$

Assignment is deterministic under a stored random seed. The complete manifest records filenames, titles, partition labels, character counts, scene counts, and sample counts. Movie-level splitting prevents temporally adjacent windows from the same screenplay from entering both training and evaluation sets.

## 8 Variational recurrent forecasting model

### 8.1 Relation to the VRAE

The variational recurrent autoencoder combines recurrent sequence encoders with stochastic variational inference [2, 4]. The referenced public implementation applies this framework to binary MIDI sequences in TensorFlow. The present model retains the recurrent variational encoding principle but adapts the decoder objective from sequence reconstruction to one-step graph forecasting. It is implemented in modern PyTorch and runs entirely on the CPU.

### 8.2 Input projection and recurrent encoder

Directly applying a large recurrent layer to a 32,896-dimensional input would be computationally expensive. Each scene vector is therefore projected to a lower-dimensional embedding:

$$e_t = f_{\text{in}}(b_t), \tag{6}$$

where  $f_{\text{in}}$  is a shared affine projection followed by layer normalization, a GELU nonlinearity, and dropout. The sequence  $(e_{s-w+1}, \dots, e_s)$  is processed by a long short-term memory network (LSTM) [3]. Let  $h_s$  denote the final encoder hidden state.

### 8.3 Latent distribution

The approximate posterior is a diagonal Gaussian

$$q_\phi(z \mid X_s) = \mathcal{N}(z; \mu_\phi(h_s), \text{diag}(\sigma_\phi^2(h_s))). \tag{7}$$

The encoder produces vectors  $\mu$  and  $\log \sigma^2$ . During training, a latent sample is obtained by the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{8}$$

At validation, test, and prediction time, the deterministic posterior mean  $z = \mu$  is used.

## 8.4 Recurrent decoder

The latent vector initializes the hidden and cell states of a decoder LSTM. A learned start token is supplied for one recurrent step, producing a decoder state  $g_s$ . An output projection maps this state to  $d$  logits:

$$\ell_s = W_{\text{out}}g_s + c_{\text{out}}. \quad (9)$$

The predicted Bernoulli probabilities are

$$\hat{p}_{s+1} = \text{sigmoid}(\ell_s). \quad (10)$$

Each coordinate estimates the probability that the corresponding diagonal character indicator or off-diagonal interaction occurs in the next scene.

The default architecture uses a 128-dimensional input embedding, a 128-unit encoder and decoder LSTM, and a 32-dimensional latent variable. These dimensions are configurable.

## 9 Training objective

### 9.1 Weighted prediction loss

Positive entries are rare relative to the 32,896 possible upper-triangular coordinates. Unweighted elementwise accuracy would therefore favor nearly all-zero predictions. Let  $\alpha$  be the ratio of negative to positive target entries in the training set, capped at a configurable maximum. The prediction term is weighted binary cross-entropy with logits:

$$\begin{aligned} \mathcal{L}_{\text{pred}} = -\frac{1}{d} \sum_{k=1}^d & \left[ \alpha y_{s,k} \log \hat{p}_{s+1,k} \right. \\ & \left. + (1 - y_{s,k}) \log(1 - \hat{p}_{s+1,k}) \right]. \end{aligned} \quad (11)$$

The weight  $\alpha$  is estimated from training movies only.

### 9.2 Kullback–Leibler regularization

The approximate posterior is regularized toward a standard normal prior. For latent dimension  $r$ ,

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2r} \sum_{j=1}^r (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2). \quad (12)$$

The complete loss is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \beta \mathcal{L}_{\text{KL}}. \quad (13)$$

The coefficient  $\beta$  increases linearly from zero to its configured maximum during an initial warm-up period. This delays strong prior regularization while the decoder first learns to use the latent representation, reducing the risk of posterior collapse.

### 9.3 Optimization

Parameters are optimized with AdamW [5]. Gradient norms are clipped, and early stopping is based on validation average precision for off-diagonal interaction entries. Training checkpoints store the model, optimizer state, configuration, validation-selected thresholds, and random seed. All tensor operations use `torch.device('cpu')`; CUDA is neither required nor assumed.

## 10 Evaluation

### 10.1 Separate prediction tasks

Metrics are reported for three coordinate sets:

1. all packed upper-triangular coordinates;
2. diagonal coordinates, which represent character appearance;
3. strict upper-triangular coordinates, which represent pairwise interaction.

This separation prevents easier character-presence predictions from masking weak performance on the primary edge-forecasting task.

### 10.2 Precision-recall metrics

Because positives are sparse, area under the precision-recall curve is emphasized over ordinary accuracy [6]. The implementation reports approximate average precision, precision, recall, and F1. It also reports  $\text{precision}@k$  and  $\text{recall}@k$  for several values of  $k$ . A decision threshold is selected on validation movies to maximize F1 and then held fixed for test evaluation.

Exact storage of every probability from every dense test vector can require substantial memory. Metrics are therefore accumulated in a configurable histogram of probability bins. Positive and negative counts are recorded separately in each bin, and approximate precision-recall curves are obtained by cumulative summation from high to low probability.

### 10.3 Baselines

The learned model is compared with four baselines:

**All zero** Predict no active characters or interactions.

**Repeat last** Set  $\hat{B}_{s+1} = B_s$ .

**Recent union** Predict the entrywise union of the five observed local matrices.

**Global frequency**

Assign each coordinate its empirical frequency among training targets.

The global-frequency threshold is chosen on validation movies. The direct binary baselines use a threshold of 0.5.

## 11 Prediction outputs

For a specified movie prefix, the prediction program returns

- the packed probability vector;
- the reconstructed symmetric  $256 \times 256$  probability matrix;
- a thresholded packed vector and symmetric binary matrix;

- a ranked list of likely next-scene character appearances;
- a ranked list of likely next-scene pairwise interactions.

Character names are recovered from the movie-specific metadata. Historical forecasts can be generated by specifying a scene index inside a completed screenplay; otherwise, the final five observed scenes are used to predict one additional scene.

## 12 Reproducibility and computational safeguards

The software records the random seed, dataset manifest, split membership, configuration, checkpoints, and evaluation output. The preprocessing pass is completed before any processed files are written, ensuring that a malformed or over-limit movie does not produce a partially valid dataset. Tests verify upper-triangle round trips, clique construction, singleton diagonals, rejection of movies with more than 256 characters, and model output dimensions.

## 13 Interpretive limitations

The scene-clique rule is a reproducible operational definition, but it treats all speaking characters in a scene as mutually interacting. It does not distinguish direct address, separate conversations within a scene, or characters who speak without acknowledging one another. Conversely, visually present nonspeaking characters are omitted. Future extensions could infer directed utterance networks, weight edges by dialogue exchange, or divide long scenes into finer temporal units.

The chronological character coordinate system is intentionally not permutation-invariant. This permits the model to learn patterns such as the introduction of a new low-index or next-unused character, but it also means that performance reflects both graph structure and narrative introduction order. A graph neural architecture with explicit permutation handling would answer a different scientific question.

Finally, the model observes only interaction structure and not dialogue text, scene headings, genre, or character attributes. Any predictive signal must therefore arise from temporal regularities in the network sequence itself.

## 14 Software organization

The implementation is divided into modules for JSON validation and graph construction, processed-dataset creation, sliding-window loading, the VRAE forecaster, losses, streaming metrics, training, evaluation, prediction, tests, and exploratory notebooks. Hyperparameters are stored in a single JSON configuration file. The complete package is designed for command-line execution on machines without a GPU.

## References

- [1] M. Bostock, V. Ogievetsky, and J. Heer. D3: Data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.
- [2] O. Fabius and J. R. van Amersfoort. Variational recurrent auto-encoders. *arXiv preprint arXiv:1412.6581*, 2014.



- [3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.
- [5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [6] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [7] A. Mittal. Variational Recurrent AutoEncoder: TensorFlow implementation on MIDI data. <https://github.com/arunesh-mittal/VariationalRecurrentAutoEncoder>. Accessed June 22, 2026.